ACTIVIDAD PLAN PADRINO

Del problema al algoritmo y del algoritmo a la estructura de datos

Estudiantes de I y IV semestre

Ingeniería de Software – Ingeniería de Sistemas

Asignatura articuladora	Estructura de Datos II – Lógica Matemática
Duración sugerida	2 horas
Modalidad	Presencial – trabajo colaborativo
Lenguaje / herramientas	Java, IDE, Draw.io y GitHub
Producto central	Directorio inteligente universitario con Árbol Binario de Búsqueda (BST)
Ingeniería de Sistemas	8 estudiantes
Ingeniería de Software	19 estudiantes
Ingeniería de Industrial	6 estudiantes
Fecha de la Actividad	26/10/2026

1. Propósito de la actividad

Integrar estudiantes de primer y cuarto semestre mediante la resolución colaborativa de un problema sencillo de Ingeniería de software e Ingeniería de Sistemas. Los estudiantes de primer semestre fortalecerán pensamiento lógico, algoritmos, modelado y pruebas; los estudiantes de cuarto semestre aplicarán estructuras de datos, programación en Java, comunicación técnica, liderazgo y mentoría.

La actividad está diseñada para que ambos grupos tengan responsabilidades reales. Los estudiantes de IV semestre no deben limitarse a programar la solución: su responsabilidad principal es acompañar, explicar, preguntar y validar que los estudiantes de I semestre comprendan el problema y puedan argumentar las decisiones tomadas.

2. Resultados esperados

- Identificar entradas, procesos y salidas de un problema computacional.
- Representar una solución mediante pseudocódigo y diagramas.
- Comprender el concepto de estructura de datos no lineal a partir de un ejemplo práctico.
- Implementar inserción, búsqueda y recorrido en un Árbol Binario de Búsqueda (BST).
- Aplicar pruebas funcionales básicas y registrar sus resultados.
- Fortalecer la comunicación técnica, el aprendizaje entre pares y el trabajo colaborativo.
- Documentar la solución y sus evidencias en Moodle y GitHub.

3. Reto de trabajo

DIRECTORIO INTELIGENTE UNIVERSITARIO

Cada equipo desarrollará un pequeño sistema para registrar y consultar elementos de la vida universitaria. El equipo puede elegir una categoría principal: estudiantes, asignaturas, libros, salones, laboratorios, docentes o equipos de cómputo.

Cada registro tendrá como mínimo los siguientes datos:

```
Código: 105
Nombre: Programación I
Categoría: Asignatura
```

El sistema debe permitir:

1. Registrar información.
2. Buscar un registro por código.
3. Mostrar todos los registros.
4. Identificar si un registro existe.
5. Eliminar un registro.
6. Mostrar los datos ordenados.

Los estudiantes de I semestre lideran el análisis del problema y el diseño lógico inicial. Los estudiantes de IV semestre orientan la transformación de esa lógica en una solución basada en BST y apoyan la implementación en Java.

4. Organización de los equipos

Se recomienda conformar equipos de 4 o 5 integrantes: 2 estudiantes de IV semestre y 2 o 3 estudiantes de I semestre.

Rol	Semestre sugerido	Responsabilidad
Analista	I	Identifica el problema, los datos, las entradas, los procesos y las salidas.
Diseñador lógico	I	Construye el pseudocódigo o diagrama de flujo inicial.
Mentor técnico	IV	Explica la estructura BST, sus nodos y sus operaciones con lenguaje claro.
Programador	IV	Orienta la implementación en Java sin sustituir el trabajo del equipo.
Tester / Documentador	I o IV	Ejecuta pruebas, registra resultados y organiza las evidencias.

Regla de la actividad: el estudiante de IV semestre no puede desarrollar solo el programa. Debe involucrar al estudiante de I semestre en cada decisión, prueba y explicación.

5. Desarrollo paso a paso

Fase 1. Rompehielo técnico

Tiempo sugerido: 10 minutos. Cada estudiante de IV semestre explica qué es una estructura de datos utilizando únicamente un ejemplo cotidiano: biblioteca, árbol genealógico, contactos del celular, carpetas del computador o catálogo de productos. Después, el equipo construye una definición sencilla y propia. No se permite iniciar con una definición memorizada.

Fase 2. Comprender el problema

Tiempo sugerido: 15 minutos. Los estudiantes de I semestre identifican qué información recibe el programa, qué debe hacer con ella y qué resultados debe mostrar. El equipo registra la información en una tabla Entrada–Proceso–Salida.

Fase 3. Diseñar el algoritmo

Tiempo sugerido: 15 minutos. Los estudiantes de I semestre elaboran un pseudocódigo básico del menú y de las operaciones principales. Los estudiantes de IV semestre revisan mediante preguntas orientadoras, sin reemplazar inmediatamente la propuesta.

Fase 4. Del algoritmo lineal al árbol

Tiempo sugerido: 15 minutos. Los estudiantes de IV semestre explican visualmente el BST y muestran cómo cada valor se ubica a la izquierda o a la derecha según su clave. Luego, un estudiante de I semestre explica con sus propias palabras por qué una estructura organizada puede mejorar la búsqueda.

Fase 5. Construcción en Java

Tiempo sugerido: 35 minutos. El equipo implementa las clases Registro, Nodo y ArbolBST. Los estudiantes de IV semestre orientan la codificación; los estudiantes de I semestre deben comprender qué representa cada clase y cada operación.

Fase 6. Prueba cruzada

Tiempo sugerido: 15 minutos. Los estudiantes de I semestre asumen el rol de testers. Deben insertar datos, buscar registros existentes e inexistentes, mostrar los datos y registrar cualquier error o comportamiento inesperado.

Fase 7. Cambio de roles y cierre

Tiempo sugerido: 10 minutos. Un estudiante de I semestre explica cómo funciona el programa. Un estudiante de IV semestre explica qué aprendió al enseñar el concepto y qué estrategia utilizó para hacerlo comprensible.

Ejemplo de análisis Entrada – Proceso – Salida

Entrada	Proceso	Salida
Código	Buscar	Registro encontrado / no encontrado
Nombre y categoría	Insertar	Registro almacenado
Código	Eliminar	Registro eliminado o mensaje de error

Ejemplo de pseudocódigo inicial

```
INICIO

Mostrar menú

1. Registrar
2. Buscar
3. Mostrar
4. Eliminar
5. Salir

Leer opción

SI opción = 1
    Leer código
    Leer nombre
    Leer categoría
    Registrar
FIN SI

SI opción = 2
    Leer código
    Buscar registro
FIN SI

FIN
```

Ejemplo visual de un BST

```
Valores de ejemplo: 50, 30, 70, 20, 40, 60, 80

           50
         /    \
       30      70
      /  \    /  \
    20   40  60   80
```

6. Código base en Java

El siguiente código es una base orientadora. El equipo debe comprenderlo, probarlo y adaptarlo al caso elegido. No se espera que los estudiantes de I semestre dominen todavía la recursividad; sí deben comprender qué representa cada clase y qué hace cada operación.

6.1. Clase Registro

```
public class Registro {

    int codigo;
    String nombre;
    String categoria;

    public Registro(int codigo, String nombre, String categoria) {
        this.codigo = codigo;
        this.nombre = nombre;
        this.categoria = categoria;
    }

    @Override
    public String toString() {
        return codigo + " - " + nombre + " - " + categoria;
    }
}
```

6.2. Clase Nodo

```
public class Nodo {

    Registro dato;
    Nodo izquierdo;
    Nodo derecho;

    public Nodo(Registro dato) {
        this.dato = dato;
    }
}
```

6.3. Clase ArbolBST: inserción

```
public class ArbolBST {

    Nodo raiz;

    public void insertar(Registro registro) {
        raiz = insertarRecursivo(raiz, registro);
    }

    private Nodo insertarRecursivo(Nodo actual, Registro registro) {

        if (actual == null) {
            return new Nodo(registro);
        }

        if (registro.codigo < actual.dato.codigo) {
            actual.izquierdo =
                insertarRecursivo(actual.izquierdo, registro);
        } else {
            actual.derecho =
                insertarRecursivo(actual.derecho, registro);
        }

        return actual;
    }
}
```

6.4. Búsqueda

```
public Registro buscar(int codigo) {
    return buscarRecursivo(raiz, codigo);
}

private Registro buscarRecursivo(Nodo nodo, int codigo) {

    if (nodo == null) {
        return null;
    }

    if (codigo == nodo.dato.codigo) {
        return nodo.dato;
    }

    if (codigo < nodo.dato.codigo) {
        return buscarRecursivo(nodo.izquierdo, codigo);
    }

    return buscarRecursivo(nodo.derecho, codigo);
}
```

6.5. Recorrido Inorden

```
public void mostrarOrdenado() {
    inOrden(raiz);
}

private void inOrden(Nodo nodo) {

    if (nodo != null) {

        inOrden(nodo.izquierdo);

        System.out.println(nodo.dato);

        inOrden(nodo.derecho);
    }
}
```

6.6. Orientación para la eliminación

La eliminación puede trabajarse como ampliación o reto de cierre. Antes de programarla, el equipo debe identificar los tres casos: nodo hoja, nodo con un hijo y nodo con dos hijos. Los estudiantes de IV semestre deben explicar cómo se conservan las referencias del árbol.

7. Prueba cruzada

Los estudiantes de I semestre deben ejecutar como mínimo las siguientes pruebas:

- Insertar 10 registros.
- Buscar un código existente.
- Buscar un código inexistente.
- Mostrar todos los registros.
- Verificar que el recorrido Inorden muestre los códigos ordenados.
- Intentar una operación inválida y registrar el comportamiento.
- Proponer un caso adicional de prueba.

Prueba	Resultado esperado	Resultado obtenido	Estado
Buscar código 50	Registro encontrado		
Buscar código 99	No encontrado		
Mostrar registros	Datos ordenados por código		
Prueba adicional			

8. Producto final del equipo

Cada equipo deberá entregar una carpeta o repositorio con una organización similar a la siguiente:

```
Actividad_Intersemestral/
│
├── README.md
├── DiagramaProblema.png
├── Pseudocodigo.pdf
│
├── src/
│   ├── Registro.java
│   ├── Nodo.java
│   ├── ArbolBST.java
│   └── Main.java
│
├── Pruebas.pdf
└── Evidencia.jpg
```

9. Reflexión obligatoria

En el archivo README.md, el equipo deberá responder:

Estudiantes de I semestre: ¿Qué aprendí hoy que todavía no conocía sobre programación?

Estudiantes de IV semestre: ¿Qué concepto fue más difícil de explicar y cómo logré explicarlo?

Equipo: ¿Cómo podría utilizarse esta solución en un problema real?

10. Evidencias para Moodle y GitHub

1. Enlace del repositorio GitHub.
2. Diagrama del problema.
3. Pseudocódigo.
4. Código Java funcional.
5. Tabla de pruebas diligenciada.
6. Captura de la ejecución del programa.
7. Fotografía o evidencia del trabajo colaborativo.
8. Reflexión conjunta de máximo una página.

11. Rúbrica de evaluación – escala de 10 a 50 puntos

La evaluación considera tanto el producto técnico como la calidad del trabajo intersemestral. La calificación se asigna de acuerdo con el descriptor que mejor represente el desempeño observado.

Criterio	10	20	30	40	50
Análisis del problema	No identifica adecuadamente el problema.	Identificación muy limitada.	Reconoce entradas y salidas.	Analiza correctamente entradas, procesos y salidas.	Analiza y justifica completamente la solución.
Diseño algorítmico	No presenta diseño.	Diseño incompleto.	Pseudocódigo básico.	Algoritmo organizado y coherente.	Algoritmo completo, claro y correctamente explicado.
Implementación Java / BST	La solución no funciona.	Presenta fallos importantes.	Funciona parcialmente.	Inserción, búsqueda y recorrido funcionan.	Solución funcional, organizada, documentada y correctamente estructurada.
Trabajo intersemestral	No existe integración.	Participación muy desigual.	Existe colaboración básica.	Ambos semestres participan activamente.	Evidencia mentoría, aprendizaje entre pares y cambio efectivo de roles.
Evidencias y sustentación	No entrega evidencias.	Evidencias incompletas.	Presenta evidencias básicas.	Evidencias completas y explicación adecuada.	GitHub organizado, pruebas completas y sustentación técnica clara.

12. Criterios de cierre para la socialización

Al finalizar, cada equipo dispondrá de 3 a 5 minutos para demostrar el sistema. La explicación debe ser realizada principalmente por un estudiante de I semestre, mientras que un estudiante de IV semestre complementará la explicación técnica del BST y describirá la estrategia de mentoría utilizada.

META DE APRENDIZAJE

Que el estudiante de I semestre pueda explicar qué problema resolvió y cómo funciona la lógica general; y que el estudiante de IV semestre pueda demostrar que comprende una estructura de datos al implementarla y enseñarla.